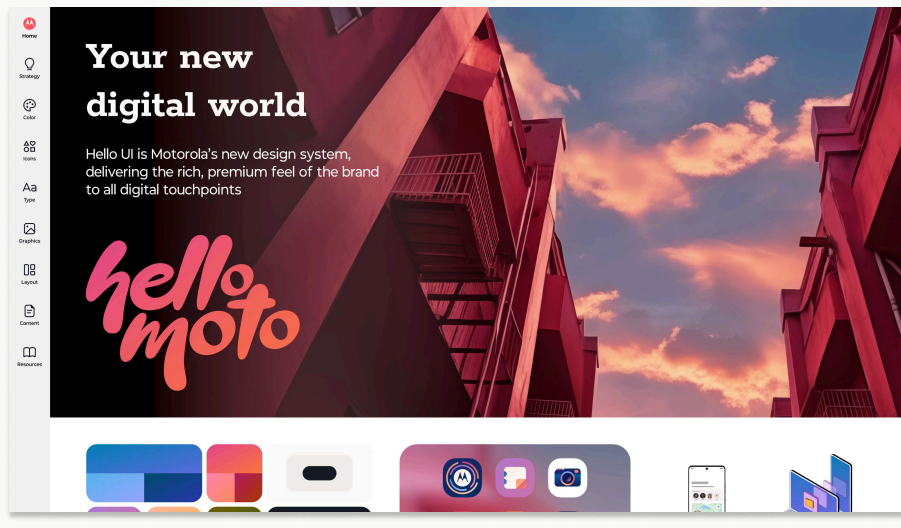


Hello UI

Construindo o design system da Motorola do zero
— e fazendo as pessoas realmente usá-lo.



CONTEXTO

Motorola — Hello UI
Pontos de contato
digitais globais

DURAÇÃO

2 meses,
da base à
v1.0

FUNÇÃO

Design
Systems
Lead Designer

STATUS

Lançado
Web +
Mobile

A TESE

**Um design system não é uma
biblioteca de componentes.**

**É um contrato que os
times escolhem honrar.**

*O Hello UI não venceu porque os
componentes eram bonitos. Venceu porque
engenharia, produto e design quiseram usá-lo.*

O QUE MUDOU

~4–5 times de produto adotando nos primeiros meses

~30–40 componentes entregues com variantes

A engenharia saiu do "por quê?" para o "o que vem agora?"

Marca sem sistema.

A Motorola tinha assets de design — não um design system. Cada time de produto interpretava a marca do seu jeito. Um app Moto podia parecer cinco empresas diferentes, dependendo de quem o desenhava.

O problema mais difícil não era visual. Era cultural. A engenharia via um novo sistema como mais trabalho sem ganho claro.

"Não precisamos de um novo design system. Precisamos de um sistema que as pessoas escolham. "

AS TRÊS TENSÕES QUE ENCONTRAMOS

- **Expressão de marca inconsistente** entre os produtos
- **Material Design** usado puro, sem identidade Moto
- **Resistência da engenharia** à adoção — e eles tinham razão

Estratégia primeiro. Pixels depois.

A maioria dos design systems pula a etapa de estratégia e acaba virando uma biblioteca de componentes glorificada. Nós alinhamos uma posição antes de desenhar um único botão.

PILAR 01

Moto integrado

Um celular Motorola deve parecer uma coisa só. De ponta a ponta. Sem emendas entre marca e sistema operacional.

PILAR 02

Sim, E...

Estendemos o Material Design. Não duplicamos nem brigamos com ele. Polimos onde importa, deixamos o resto em paz.

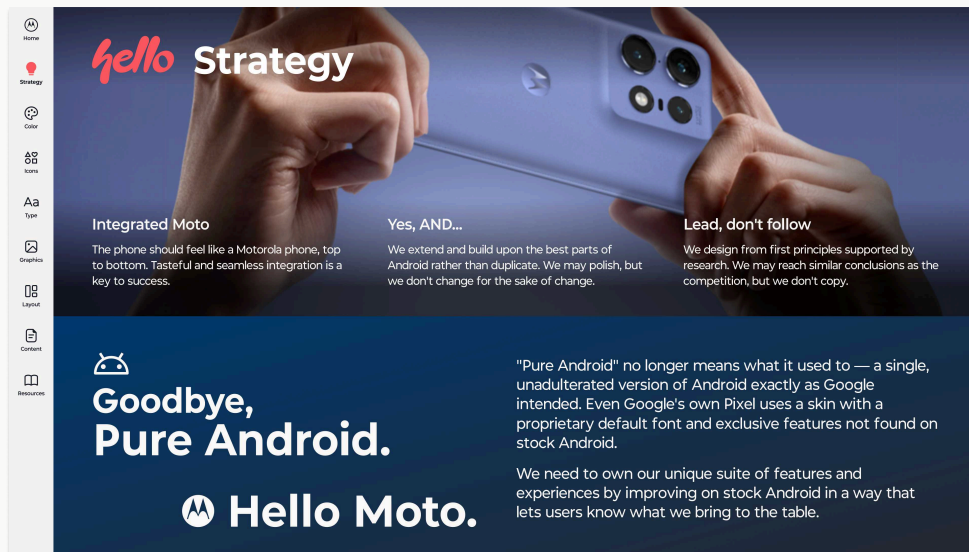
PILAR 03

Liderar, não seguir

Projetamos a partir de princípios fundamentais. Podemos chegar às mesmas conclusões que concorrentes.

Desses três pilares vieram quatro princípios de design — Humano, Confiante, Autêntico, Cuidado — que se tornaram a lente para toda decisão seguinte.

A estratégia vive dentro do sistema — não num deck que ninguém lê.



Todo time que usa o Hello UI vê a estratégia logo na primeira página do sistema. Não é aspiracional — é o contrato em vigor.

- **3 pilares** definem o posicionamento
- **4 princípios de design** transformam o posicionamento em decisões
- Todo componente, token e template é avaliado em relação a eles

Quando alguém propunha um novo padrão, a pergunta passava a ser: **"Isto é Confiante? Isto é Cuidado?"**

Tokens são a língua em que design e engenharia falam juntos.

Construímos uma hierarquia de quatro níveis para que um designer nunca pense em códigos hex e um engenheiro nunca fixe um valor no código.

- **Valor primitivo** - hex bruto / HCT
- **Token de marca** - Whale 90
- **Token de sistema** - Primary 90
- **Token de tema (papel de cor)** - Primary container

Mude uma cor de marca em um único lugar. Ela se propaga por todas as telas, todos os temas, todos os produtos. O contraste WCAG AA foi incorporado desde o primeiro dia — não remendado no fim.

Token hierarchy

There are different types of tokens that can point to one another:

Primitive value

The final static color value (Hex / HCT)

Brand token

The value of a brand color

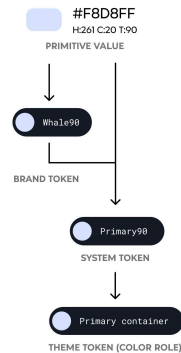
System token

Used to link a color to the dynamic color system and comprises all of the color options available in the design system

Theme token (color role)

Define the purpose that a reference token serves in the UI

[Access the Figma library →](#)



Estenda o Android. Não brigue com ele.

Theme comparison

The Moto theme uses Vellum for both neutral ranges and customized tonal values for some tokens.

	Moto Light	Material Light	Moto Dark	Material Dark
	ACCENT COLORS	ACCENT COLORS	ACCENT COLORS	ACCENT COLORS
PRIMARY	PRIMARY	PRIMARY	PRIMARY	PRIMARY
	ON PRIMARY	ON PRIMARY	ON PRIMARY	ON PRIMARY
	PRIMARY CONTAINER	PRIMARY CONTAINER	PRIMARY CONTAINER	PRIMARY CONTAINER
	ON PRIMARY CONTAINER	ON PRIMARY CONTAINER	ON PRIMARY CONTAINER	ON PRIMARY CONTAINER
SECONDARY	SECONDARY	SECONDARY	SECONDARY	SECONDARY
	ON SECONDARY	ON SECONDARY	ON SECONDARY	ON SECONDARY
	SECONDARY CONTAINER	SECONDARY CONTAINER	SECONDARY CONTAINER	SECONDARY CONTAINER
	ON SECONDARY CONTAINER	ON SECONDARY CONTAINER	ON SECONDARY CONTAINER	ON SECONDARY CONTAINER
TERTIARY	TERTIARY	TERTIARY	TERTIARY	TERTIARY
	ON TERTIARY	ON TERTIARY	ON TERTIARY	ON TERTIARY
	TERTIARY CONTAINER	TERTIARY CONTAINER	TERTIARY CONTAINER	TERTIARY CONTAINER
	ON TERTIARY CONTAINER	ON TERTIARY CONTAINER	ON TERTIARY CONTAINER	ON TERTIARY CONTAINER
PRIMARY FIXED	PRIMARY FIXED	PRIMARY FIXED	PRIMARY FIXED	PRIMARY FIXED
	ON PRIMARY FIXED	ON PRIMARY FIXED	ON PRIMARY FIXED	ON PRIMARY FIXED
	ON PRIMARY FIXED VARIANT	ON PRIMARY FIXED VARIANT	ON PRIMARY FIXED VARIANT	ON PRIMARY FIXED VARIANT
	SECONDARY FIXED	SECONDARY FIXED	SECONDARY FIXED	SECONDARY FIXED
SECONDARY FIXED	ON SECONDARY FIXED	ON SECONDARY FIXED	ON SECONDARY FIXED	ON SECONDARY FIXED
	ON SECONDARY FIXED VARIANT	ON SECONDARY FIXED VARIANT	ON SECONDARY FIXED VARIANT	ON SECONDARY FIXED VARIANT
	TERTIARY FIXED	TERTIARY FIXED	TERTIARY FIXED	TERTIARY FIXED
	TERTIARY FIXED DIM	TERTIARY FIXED DIM	TERTIARY FIXED DIM	TERTIARY FIXED DIM
TERTIARY FIXED	ON TERTIARY FIXED	ON TERTIARY FIXED	ON TERTIARY FIXED	ON TERTIARY FIXED
	ON TERTIARY FIXED VARIANT	ON TERTIARY FIXED VARIANT	ON TERTIARY FIXED VARIANT	ON TERTIARY FIXED VARIANT
INVERSE	INVERSE PRIMARY	INVERSE PRIMARY	INVERSE PRIMARY	INVERSE PRIMARY

O Android usa o algoritmo Monet, do Google, para gerar temas a partir do wallpaper do usuário. Nós o adaptamos — mas customizamos as faixas neutras com uma cor Moto chamada **Vellum**.

Resultado: o dispositivo parece nitidamente Moto mesmo quando personalizado. O custo de engenharia permaneceu baixo porque a estrutura subjacente era familiar.

Isto foi o "Sim, E" na prática. Não reinventamos a cor dinâmica. Tornamos ela nossa onde fazia diferença, mantivemos o Material onde funcionava.

Telas não são superfícies genéricas. Elas têm funções.

Um erro comum em design systems é tratar toda tela igual. Mapeamos três momentos centrais — cada um com suas próprias qualidades, layouts e templates.

MOMENTO 01

Descobrir

Objetivo do usuário: explorar funcionalidades, agir.

Qualidades: impactante, editorial, guiado por imagem.

Exemplos: propostas de valor, telas de abertura, tutoriais.

MOMENTO 02

Iniciar

Objetivo do usuário: encontrar e entrar em experiências.

Qualidades: atmosférico, imersivo, infinito.

Exemplos: tela inicial, tela de bloqueio, gaveta de apps.

MOMENTO 03

Configurar

Objetivo do usuário: ajustar configurações, ativar features

Qualidades: minimalista, alto contraste, sem ruído.

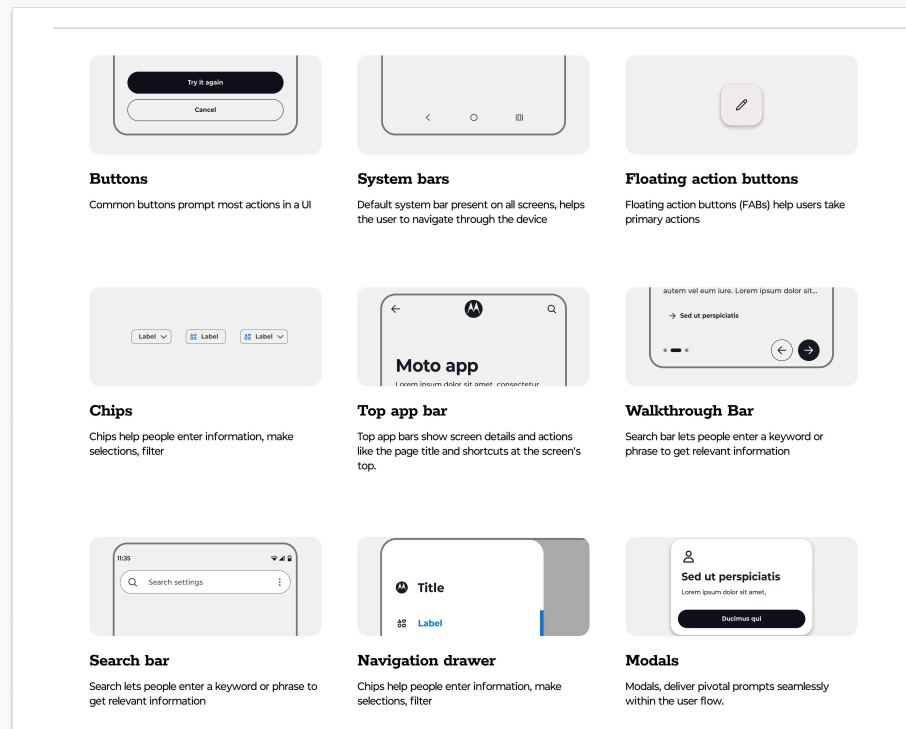
Exemplos: configurações do sistema, dashboards, páginas de índice.

Os times pararam de perguntar "**qual botão eu uso?**" e passaram a perguntar "**para qual momento estou desenhando?**"
Essa mudança transformou como a organização pensava em consistência.

Uma biblioteca que cresceu com o produto — não contra ele.

Os componentes foram construídos no Figma com variáveis, variantes e propriedades. Os templates, organizados por momento, para que designers começassem a partir de uma base, não de uma tela em branco.

- **~30–40 componentes** com variantes até a v1.0
- **500+ ícones de sistema** em 4 tamanhos otimizados
- **Templates editáveis** por momento, prontos para derivar
- **Cobertura web + mobile** a partir da mesma fonte de verdade



Construir o sistema é a metade fácil. Fazer as pessoas usarem é o trabalho de verdade.

A resistência da engenharia era real. E justa. Por isso tratamos a adoção como um problema de design, não de marketing.

MOVIMENTO 01

Arquitetura de biblioteca em três níveis

Biblioteca de marca → Estilos e Componentes → Templates. As pessoas encontravam o que precisavam sem se perder em um arquivo gigante.

MOVIMENTO 02

Modelo de contribuição embutido

Um sistema de "Fazer um Pedido" dentro das diretrizes. Designers e engenheiros podiam apontar lacunas e propor mudanças. O sistema deixou de ser só nosso.

MOVIMENTO 03

Construído sobre o Material, não contra ele

O maior destravamento. Os engenheiros já dominavam o Material. Estendemos o que eles conheciam, em vez de pedir que aprendessem um mundo novo.

Workflows assistidos por IA mudam o que um systems designer pode dominar.

Tenho explorado ativamente **servidores MCP**, **Cursor** e automação de design-to-code em projetos pessoais.

Ainda não entreguei código de produção com eles. Mas já vi o suficiente para acreditar que isso desloca a fronteira entre design e implementação — e é exatamente essa fronteira que quero continuar cruzando.

O QUE JÁ EXPERIMENTEI

- **Configurações de servidor MCP** para sincronizar tokens entre Figma e código
- **Cursor / Claude Code** para geração e refatoração de componentes
- **Protótipos design-to-code** explorando integração com o Storybook

COMO EU ENXERGO ISSO

Sendo honesto: sou um designer que entende implementação, não um engenheiro. O crescimento que busco é me aprofundar no código ao lado dos engenheiros — é isso que faz essa função fazer sentido para mim.

Estávamos no começo. Os números são direcionais. A mudança na sala não foi.

~4–5

Times de produto adotando nos primeiros meses

Medido pelo uso da biblioteca no Figma

~30–40

Componentes com variantes entregues até a v1.0

Entre web e mobile

~50%

Tempo economizado em telas rotineiras, segundo os designers

Anedótico — sem medição formal de tempo

500+

Ícones de sistema em 4 tamanhos otimizados

Conjuntos preenchidos + contornados

A mudança maior foi cultural. As conversas com a engenharia saíram de **"por que estamos fazendo isso?"** para **"quando vem o próximo lote de tokens?"**

Isso, para mim, é o verdadeiro ROI de um design system.

Dois atritos que eu resolveria mais cedo da próxima vez.

ATRITO 01

Instrumentar as métricas de adoção desde o primeiro dia.

Medimos tarde demais. Uso da biblioteca, time-to-ship, dívida de design — isso deveria estar conectado antes de o sistema ser lançado, não depois da primeira onda de adoção.

ATRITO 02

Trazer a engenharia para a fase de estratégia, não só para a implementação.

A co-propriedade precisa começar antes da primeira linha de código. Fizemos isso bem na execução. Mais cedo teria facilitado o buy-in.

O julgamento sênior não é entregar o perfeito. É saber quais imperfeições são seguras de entregar — e quais você resolveria mais cedo na próxima vez.

EXPLORE O PROTÓTIPO DAS DIRETRIZES

[Link do protótipo de diretrizes no Figma](#)

Obrigado pela leitura.